

Reporte de Proyecto Final: Semáforos en la FPGA Nexys A7

Daniela Ulloa Barboza, B77748
Luis Diego Núñez Mora, B75473.

2 de diciembre de 2024

Resumen

Este documento se encarga de explicar el código necesario para poder ejecutar el presente proyecto que consiste en la emulación de una red de semáforos en una RVFPGA marca Digilent Nexys A7. Es un programa cíclico que funciona a partir de una máquina de estados y una interrupción; el objetivo del proyecto es demostrar en tiempo real y de una forma gráfica cómo ocurre una Interrupción, y la implementación de una Máquina de estados en el lenguaje de programación C (tema del curso a implementar). El código funciona de la siguiente manera, un LED tricolor cambia constantemente su color (rojo - verde - amarillo) simulando que es parte de una red de semáforos en una intersección mientras ocurre un conteo inverso en los display de 7 segmentos por cada color, pero además se agrega un semáforo de emergencia (Interrupción) que detenga el funcionamiento cíclico y que mientras el de emergencia esté encendido el otro se mantengan en rojo y detenido; el semáforo de emergencia representaría en realidad, una vía de un tren y las señales correspondientes para activar y desactivar las agujas, por lo que si la luz está en rojo las agujas del tren estarán bloqueadas y si la luz está en verde es que bajan las agujas del tren. La interrupción se generará cada vez que se accione el botón central de los Push-Buttons que trae la Nexys, el accionamiento del botón representa el acercamiento del tren a la intersección, esto se concreta utilizando los registros de interrupción de la PTC (PWM/-Temporizador/Contador por sus siglas en inglés) de la tarjeta RVFPGA Digilent Nexys A7 y la programación ya brindada para llevar a cabo las interrupciones, también se hará uso de la programación necesaria para acceder a los LEDs tricolor. Otro objetivo del proyecto sería exponer a los compañeros el proceso de cómo hacer las conexiones del hardware, programando con Verilog, para que así se pueda también comprender un poco la arquitectura interna de la FPGA Nexys, una tarjeta altamente usada en el ambiente laboral. Para poder acceder al Overleaf, por favor acceder al siguiente link: <https://www.overleaf.com/read/vkdssxhsxmpc#83a7b2>.

Índice

1. Análisis del programa	3
1.1. Cambios en los archivos de interpretación del Hardware	3
1.2. Máquina de Estados para un Semáforo	11
1.2.1. Estados y Descripciones	12
1.2.2. Diagrama de Estados	12
1.2.3. Código C Pseudodiseñado	12
1.2.4. Explicación del Funcionamiento	13
1.3. Software	13
1.3.1. Descripción del código	13
1.3.2. Librerías y definiciones iniciales	13
1.3.3. Enumeración para los estados del semáforo	14
1.3.4. Función para el manejo del display de 7 segmentos	14
1.3.5. Manejador de interrupciones	14
1.3.6. Estado del semáforo	16
1.3.7. Función principal	17
2. Conclusiones y Recomendaciones	18

Índice de figuras

1. Arquitectura de la Nexys para el proyecto de Semáforos.	3
2. Cambios en el archivo rvfpganexys.xdc.	3
3. Cambio 1 en el archivo wb_intercon.v.	4
4. Cambio 2 en el archivo wb_intercon.v.	5
5. Cambio 3 en el archivo wb_intercon.v.	5
6. Cambio 4 en el archivo wb_intercon.v.	5
7. Cambio 5 en el archivo wb_intercon.v.	6
8. Cambio 1 en el archivo wb_intercon.vh.	7
9. Cambio 2 en el archivo wb_intercon.vh.	7
10. Cambio 3 en el archivo wb_intercon.vh.	8
11. Cambio 4 en el archivo wb_intercon.vh.	8
12. Cambio 1 en el archivo swervolf_core.v.	8
13. Cambio 2 en el archivo swervolf_core.v.	9
14. Cambio 3 en el archivo swervolf_core.v.	9
15. Cambio 4 en el archivo swervolf_core.v.	10
16. Cambio 1 en el archivo rvfpganexys.sv.	10
17. Cambio 2 en el archivo rvfpganexys.sv.	10
18. Cambio 1 en el archivo rvfpgasim.v.	11
19. Cambio 2 en el archivo rvfpgasim.v.	11
20. Cambio 3 en el archivo rvfpgasim.v.	11

1. Análisis del programa

1.1. Cambios en los archivos de interpretación del Hardware

Para poder realizar el proyecto con ambos LEDs tricolor se tiene una limitante y es que solo se tiene acceso a uno de los LEDs RGB, y no es tan sencillo como solo encenderlos en software sino que se deben editar múltiples archivos que son la interpretación del hardware y arquitectura de la tarjeta, ya que el otro no posee las conexiones arquitectónicas necesarias, ni el mapeo a memoria para poder hacerlo funcionar. Por lo que a continuación se muestra paso a paso lo realizado, es muy importante que la arquitectura que se desea desarrollar para este proyecto en la Nexys, es el siguiente:

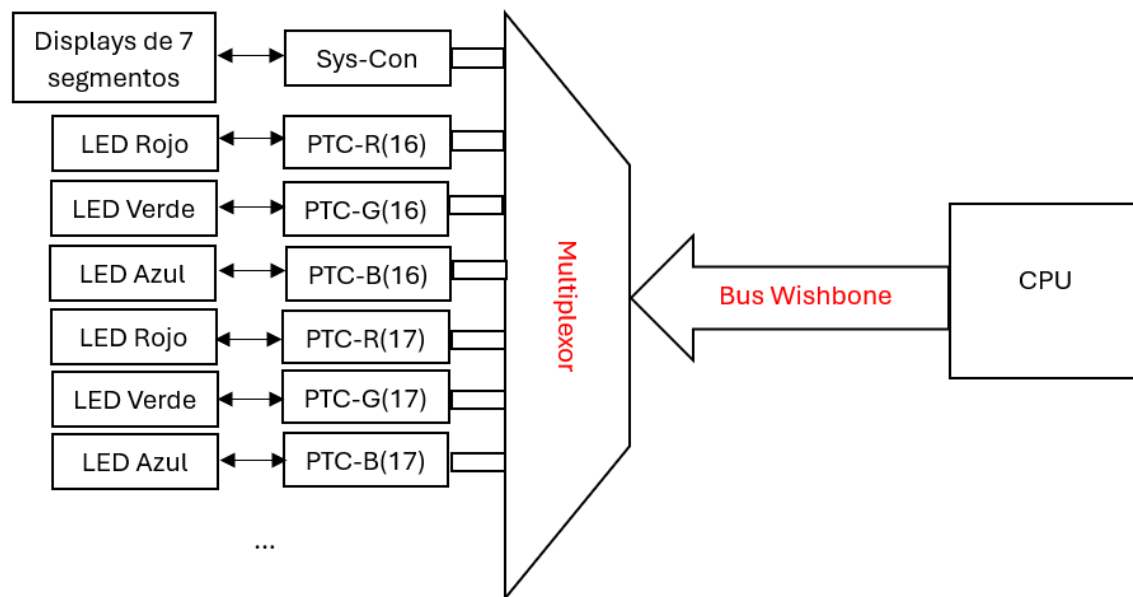


Figura 1: Arquitectura de la Nexys para el proyecto de Semáforos.

Para poder usar el segundo LED tricolor que será el LD17, se debe agregar una PTC (PWM/-Temporizador/Contador por sus siglas en inglés), el primer cambio ocurre en el documento rvfpga-nexys.xdc donde se agregan los puertos del LED tricolor y se define el wire inout i_led17 con el que se puede acceder a ellos. Se muestran a continuación:

```
# LEDs RGB Proyecto Estructuras / LED Tricolor LD17:
set_property -dict { PACKAGE_PIN N16   IOSTANDARD LVCMOS33 } [get_ports { i_led17[0] }]; # R
set_property -dict { PACKAGE_PIN R11   IOSTANDARD LVCMOS33 } [get_ports { i_led17[1] }]; # G
set_property -dict { PACKAGE_PIN G14   IOSTANDARD LVCMOS33 } [get_ports { i_led17[2] }]; # B
```

Figura 2: Cambios en el archivo rvfpganexys.xdc.

Luego se hacen los cambios respectivos en los archivos del bus Wishbone, primero los cambios del archivo wb_intercon.v, en la figuras 3 y 4 se definen los outputs e inputs de las PTC de cada LED (Red, Green, Blue) necesarios para poder utilizar el bus Wishbone y así conectarse con el CPU del computador. En la figura 5 se muestra como el multiplexor se le aumentan la cantidad de slaves o

periféricos a los que puede acceder, antes eran 11 pero ahora se le suman 3 de la PTCR2, PTCG2 y la PTCB2, por lo que ahora son 14, también sirve de evidencia que los números mostrados en la figura 6 son los espacios del mapa de memoria que se le asigna para la implementación de los registros de las nuevas PTCs que corresponden las direcciones a las que se debe hacer match para acceder (.MATCH_ADDR) y la máscara que define el límite del espacio asignado para ese periférico por medio de una máscara (.MATCH_MASK), es por esto que en la figura 6 se muestra la dirección para cada PTC2 R, G y B, respectivamente, de los LEDs tricolor y se aplica la misma máscara implementada para la PTC original y las GPIO. Por último, también es necesario ubicar en orden todas las nuevas señales del Wishbone de cada PTC como se muestra en la figura 7.

```
// PTC_R2
output [31:0] wb_ptcR2_adr_o,
output [31:0] wb_ptcR2_dat_o,
output  [3:0] wb_ptcR2_sel_o,
output      wb_ptcR2_we_o,
output      wb_ptcR2_cyc_o,
output      wb_ptcR2_stb_o,
output [2:0] wb_ptcR2_cti_o,
output [1:0] wb_ptcR2_bte_o,
input  [31:0] wb_ptcR2_dat_i,
input      wb_ptcR2_ack_i,
input      wb_ptcR2_err_i,
input      wb_ptcR2_rty_i,

// PTC_G2
output [31:0] wb_ptcG2_adr_o,
output [31:0] wb_ptcG2_dat_o,
output  [3:0] wb_ptcG2_sel_o,
output      wb_ptcG2_we_o,
output      wb_ptcG2_cyc_o,
output      wb_ptcG2_stb_o,
output [2:0] wb_ptcG2_cti_o,
output [1:0] wb_ptcG2_bte_o,
input  [31:0] wb_ptcG2_dat_i,
input      wb_ptcG2_ack_i,
input      wb_ptcG2_err_i,
input      wb_ptcG2_rty_i,
```

Figura 3: Cambio 1 en el archivo wb_intercon.v.

```
// PTC_B2
output [31:0] wb_ptcB2_adr_o,
output [31:0] wb_ptcB2_dat_o,
output  [3:0] wb_ptcB2_sel_o,
output      wb_ptcB2_we_o,
output      wb_ptcB2_cyc_o,
output      wb_ptcB2_stb_o,
output [2:0] wb_ptcB2_cti_o,
output [1:0] wb_ptcB2_bte_o,
input  [31:0] wb_ptcB2_dat_i,
input      wb_ptcB2_ack_i,
input      wb_ptcB2_err_i,
input      wb_ptcB2_rty_i,
```

Figura 4: Cambio 2 en el archivo wb_intercon.v.

```
wb_mux
#(.num_slaves (14),
  .MATCH_ADDR ({32'h00000000,
  .MATCH_MASK ({32'hffffff00,
```

Figura 5: Cambio 3 en el archivo wb_intercon.v.

```
32'h00001340, 32'h00001380, 32'h000013C0,
32'hfffffff0, 32'hfffffff0, 32'hfffffff0,
```

Figura 6: Cambio 4 en el archivo wb_intercon.v.

```
wb_ptcR2_adr_o, wb_ptcG2_adr_o, wb_ptcB2_adr_o,  
wb_ptcR2_dat_o, wb_ptcG2_dat_o, wb_ptcB2_dat_o,  
wb_ptcR2_sel_o, wb_ptcG2_sel_o, wb_ptcB2_sel_o,  
wb_ptcR2_we_o, wb_ptcG2_we_o, wb_ptcB2_we_o,  
wb_ptcR2_cyc_o, wb_ptcG2_cyc_o, wb_ptcB2_cyc_o,  
wb_ptcR2_stb_o, wb_ptcG2_stb_o, wb_ptcB2_stb_o,  
wb_ptcR2_cti_o, wb_ptcG2_cti_o, wb_ptcB2_cti_o,  
wb_ptcR2_bte_o, wb_ptcG2_bte_o, wb_ptcB2_bte_o,  
wb_ptcR2_dat_i, wb_ptcG2_dat_i, wb_ptcB2_dat_i,  
wb_ptcR2_ack_i, wb_ptcG2_ack_i, wb_ptcB2_ack_i,  
wb_ptcR2_err_i, wb_ptcG2_err_i, wb_ptcB2_err_i,  
wb_ptcR2_rty_i, wb_ptcG2_rty_i, wb_ptcB2_rty_i,
```

Figura 7: Cambio 5 en el archivo wb_intercon.v.

Ahora se procede a mostrar los cambios del archivo wb_intercon.vh, en este archivo se instancian las señales de las PTCs creadas en el archivo anterior por eso en las figuras 8 y 9 se declaran los wires que recibirán los datos provenientes de las señales del Wishborne y así se podrán diferenciar las señales que viajan del master al slave y las que van del slave al master, son el puente o bus de comunicación. Por eso en las figuras 10 y 11 se asignan las señales para cada PTC de las instancias.

Luego se edita el archivo swervolf_core.v, en este se unen las señales creadas en el archivo anterior, se crean los wires que comunican y brindan la información de la interrupción (irq), el PWM y el output enable como son buses bidireccionales, además, se asignan los puertos declarados en la figura 2 que se acceden por medio de la señal i_led17 que en este archivo se declara como inout wire pero en realidad se sabe que se utilizará solo como salida ya que los LEDs no generan señales de entrada hacia la PC, en la figura 12 es donde se muestra la declaración del inout wire y en las figuras 13, 14 y 15 es donde ocurren las uniones de las señales y donde cada puerto de cada LED se le asigna a su propio PTC.

```
// PTC_R2
wire [31:0] wb_m2s_ptcR2_adr;
wire [31:0] wb_m2s_ptcR2_dat;
wire [3:0] wb_m2s_ptcR2_sel;
wire      wb_m2s_ptcR2_we;
wire      wb_m2s_ptcR2_cyc;
wire      wb_m2s_ptcR2_stb;
wire [2:0] wb_m2s_ptcR2_cti;
wire [1:0] wb_m2s_ptcR2_bte;
wire [31:0] wb_s2m_ptcR2_dat;
wire      wb_s2m_ptcR2_ack;
wire      wb_s2m_ptcR2_err;
wire      wb_s2m_ptcR2_rty;

// PTC_G2
wire [31:0] wb_m2s_ptcG2_adr;
wire [31:0] wb_m2s_ptcG2_dat;
wire [3:0] wb_m2s_ptcG2_sel;
wire      wb_m2s_ptcG2_we;
wire      wb_m2s_ptcG2_cyc;
wire      wb_m2s_ptcG2_stb;
wire [2:0] wb_m2s_ptcG2_cti;
wire [1:0] wb_m2s_ptcG2_bte;
wire [31:0] wb_s2m_ptcG2_dat;
wire      wb_s2m_ptcG2_ack;
wire      wb_s2m_ptcG2_err;
wire      wb_s2m_ptcG2_rty;
```

Figura 8: Cambio 1 en el archivo wb_intercon.vh.

```
// PTC_B2
wire [31:0] wb_m2s_ptcB2_adr;
wire [31:0] wb_m2s_ptcB2_dat;
wire [3:0] wb_m2s_ptcB2_sel;
wire      wb_m2s_ptcB2_we;
wire      wb_m2s_ptcB2_cyc;
wire      wb_m2s_ptcB2_stb;
wire [2:0] wb_m2s_ptcB2_cti;
wire [1:0] wb_m2s_ptcB2_bte;
wire [31:0] wb_s2m_ptcB2_dat;
wire      wb_s2m_ptcB2_ack;
wire      wb_s2m_ptcB2_err;
wire      wb_s2m_ptcB2_rty;
```

Figura 9: Cambio 2 en el archivo wb_intercon.vh.

```
// PTC_R2
.wb_ptcR2_adr_o      (wb_m2s_ptcR2_adr),
.wb_ptcR2_dat_o      (wb_m2s_ptcR2_dat),
.wb_ptcR2_sel_o      (wb_m2s_ptcR2_sel),
.wb_ptcR2_we_o       (wb_m2s_ptcR2_we),
.wb_ptcR2_cyc_o      (wb_m2s_ptcR2_cyc),
.wb_ptcR2_stb_o      (wb_m2s_ptcR2_stb),
.wb_ptcR2_cti_o      (wb_m2s_ptcR2_cti),
.wb_ptcR2_bte_o      (wb_m2s_ptcR2_bte),
.wb_ptcR2_dat_i      (wb_s2m_ptcR2_dat),
.wb_ptcR2_ack_i      (wb_s2m_ptcR2_ack),
.wb_ptcR2_err_i      (wb_s2m_ptcR2_err),
.wb_ptcR2_rty_i      (wb_s2m_ptcR2_rty),

// PTC_G2
.wb_ptcG2_adr_o      (wb_m2s_ptcG2_adr),
.wb_ptcG2_dat_o      (wb_m2s_ptcG2_dat),
.wb_ptcG2_sel_o      (wb_m2s_ptcG2_sel),
.wb_ptcG2_we_o       (wb_m2s_ptcG2_we),
.wb_ptcG2_cyc_o      (wb_m2s_ptcG2_cyc),
.wb_ptcG2_stb_o      (wb_m2s_ptcG2_stb),
.wb_ptcG2_cti_o      (wb_m2s_ptcG2_cti),
.wb_ptcG2_bte_o      (wb_m2s_ptcG2_bte),
.wb_ptcG2_dat_i      (wb_s2m_ptcG2_dat),
.wb_ptcG2_ack_i      (wb_s2m_ptcG2_ack),
.wb_ptcG2_err_i      (wb_s2m_ptcG2_err),
.wb_ptcG2_rty_i      (wb_s2m_ptcG2_rty),
```

Figura 10: Cambio 3 en el archivo wb.intercon.vh.

```
// PTC_B2
.wb_ptcB2_adr_o      (wb_m2s_ptcB2_adr),
.wb_ptcB2_dat_o      (wb_m2s_ptcB2_dat),
.wb_ptcB2_sel_o      (wb_m2s_ptcB2_sel),
.wb_ptcB2_we_o       (wb_m2s_ptcB2_we),
.wb_ptcB2_cyc_o      (wb_m2s_ptcB2_cyc),
.wb_ptcB2_stb_o      (wb_m2s_ptcB2_stb),
.wb_ptcB2_cti_o      (wb_m2s_ptcB2_cti),
.wb_ptcB2_bte_o      (wb_m2s_ptcB2_bte),
.wb_ptcB2_dat_i      (wb_s2m_ptcB2_dat),
.wb_ptcB2_ack_i      (wb_s2m_ptcB2_ack),
.wb_ptcB2_err_i      (wb_s2m_ptcB2_err),
.wb_ptcB2_rty_i      (wb_s2m_ptcB2_rty),
```

Figura 11: Cambio 4 en el archivo wb.intercon.vh.

```
inout wire [2:0]  i_led16,
inout wire [2:0]  i_led17,
```

Figura 12: Cambio 1 en el archivo swervolf_core.v.

```
// PTC_R2
wire      ptcR2_irq;
wire      pwm_ptcR2;
wire      oe_ptcR2;

bidirec ptcR2 (.oe(oe_ptcR2), .inp(pwm_ptcR2), .outp( ), .bidir(i_led17[0]));

ptc_top timer_ptcR2(
    .wb_clk_i      (clk),
    .wb_rst_i      (wb_rst),
    .wb_cyc_i      (wb_m2s_ptcR2_cyc),
    .wb_adr_i      ({2'b0,wb_m2s_ptcR2_adr[5:2],2'b0}),
    .wb_dat_i      (wb_m2s_ptcR2_dat),
    .wb_sel_i      (4'b1111),
    .wb_we_i      (wb_m2s_ptcR2_we),
    .wb_stb_i      (wb_m2s_ptcR2_stb),
    .wb_dat_o      (wb_s2m_ptcR2_dat),
    .wb_ack_o      (wb_s2m_ptcR2_ack),
    .wb_err_o      (wb_s2m_ptcR2_err),
    .wb_inta_o     (ptcR2_irq),
    // External PTC Interface
    .gate_clk_pad_i (),
    .capt_pad_i    (),
    .pwm_pad_o     (pwm_ptcR2),
    .oen_padoen_o  (oe_ptcR2)
);
```

Figura 13: Cambio 2 en el archivo swervolf_core.v.

```
// PTC_G2
wire      ptcG2_irq;
wire      pwm_ptcG2;
wire      oe_ptcG2;

bidirec ptcG2 (.oe(oe_ptcG2), .inp(pwm_ptcG2), .outp( ), .bidir(i_led17[1]));

ptc_top timer_ptcG2(
    .wb_clk_i      (clk),
    .wb_rst_i      (wb_rst),
    .wb_cyc_i      (wb_m2s_ptcG2_cyc),
    .wb_adr_i      ({2'b0,wb_m2s_ptcG2_adr[5:2],2'b0}),
    .wb_dat_i      (wb_m2s_ptcG2_dat),
    .wb_sel_i      (4'b1111),
    .wb_we_i      (wb_m2s_ptcG2_we),
    .wb_stb_i      (wb_m2s_ptcG2_stb),
    .wb_dat_o      (wb_s2m_ptcG2_dat),
    .wb_ack_o      (wb_s2m_ptcG2_ack),
    .wb_err_o      (wb_s2m_ptcG2_err),
    .wb_inta_o     (ptcG2_irq),
    // External PTC Interface
    .gate_clk_pad_i (),
    .capt_pad_i    (),
    .pwm_pad_o     (pwm_ptcG2),
    .oen_padoen_o  (oe_ptcG2)
);
```

Figura 14: Cambio 3 en el archivo swervolf_core.v.

```
// PTC_B2
wire    ptcB2_irq;
wire    pwm_ptcB2;
wire    oe_ptcB2;

bidirec ptcB2 (.oe(oe_ptcB2), .inp(pwm_ptcB2), .outp( ), .bidir(i_led17[2]));

ptc_top timer_ptcB2(
    .wb_clk_i    (clk),
    .wb_rst_i    (wb_rst),
    .wb_cyc_i    (wb_m2s_ptcB2_cyc),
    .wb_adr_i    ({2'b0,wb_m2s_ptcB2_adr[5:2],2'b0}),
    .wb_dat_i    (wb_m2s_ptcB2_dat),
    .wb_sel_i    (4'b1111),
    .wb_we_i    (wb_m2s_ptcB2_we),
    .wb_stb_i    (wb_m2s_ptcB2_stb),
    .wb_dat_o    (wb_s2m_ptcB2_dat),
    .wb_ack_o    (wb_s2m_ptcB2_ack),
    .wb_err_o    (wb_s2m_ptcB2_err),
    .wb_inta_o   (ptcB2_irq),
    // External PTC Interface
    .gate_clk_pad_i ( ),
    .capt_pad_i ( ),
    .pwm_pad_o (pwm_ptcB2),
    .oen_padoen_o (oe_ptcB2)
);
```

Figura 15: Cambio 4 en el archivo swervolf_core.v.

En el archivo rvfpganexys.sv se crea la señal `i_led17` que es la que se utiliza en el archivo rvfpganexys.xdc y donde se da la conexión faltante por medio de instancias para concretar la conexión tanto de los puertos con todo el resto del sistema. Por eso en la figura 16 se crea la señal y en la figura 17 se genera la unión.

```
inout wire [2:0]  i_led16,
inout wire [2:0]  i_led17,
```

Figura 16: Cambio 1 en el archivo rvfpganexys.sv.

```
.i_led16      (i_led16[2:0]),
.i_led17      (i_led17[2:0]),
```

Figura 17: Cambio 2 en el archivo rvfpganexys.sv.

Por último, se edita el archivo rvfpgasim.v para que queden las señales agregadas y listas para posibles simulaciones, pero en realidad no se realiza ninguna simulación en esta práctica, sino que se ajusta el hardware por medio de programación para garantizar la conexión y su acceso (LED tricolor) y de una vez se aplica Vivado para generar el nuevo bitstream. Si se quisiera realizar una simulación y generar el trace.vcd, se debe editar también el archivo tb.cpp que es el testbench de

simulación, por lo tanto, como plus, en la figura 18 se declaran las señales individuales de cada LED de los tricolor, en la figura 19 se declara la señal que es la que traerá el valor de todo el sistema que fue creado de comunicación por eso se denomina igual `i_led17` y se le asignan los valores a cada LED R, G y B para simular la conexión física con los LEDs de la placa y por último se da la instancia de la señal `i_led17` de este archivo con la general de todo el sistema, esto sucede en la figura 20.

```
inout wire i_led16R,
inout wire i_led16G,
inout wire i_led16B,
inout wire i_led17R,
inout wire i_led17G,
inout wire i_led17B
```

Figura 18: Cambio 1 en el archivo `rvfpgasim.v`.

```
wire [15:0] i_sw;
wire [4:0] i_btn;
wire [2:0] i_led16;
wire [2:0] i_led17;
// assign i_sw = 16'hFEDC;
// Se asigna 0xFE34 menos el bit 0:
assign i_sw = {15'b111111100011010,i_sw0};
assign i_btn = {i_btn0, i_btn1, i_btn2, i_btn3, i_btn4};
assign i_led16 = {i_led16R, i_led16G, i_led16B};
assign i_led17 = {i_led17R, i_led17G, i_led17B};
```

Figura 19: Cambio 2 en el archivo `rvfpgasim.v`.

```
.io_data      ({i_btn,i_sw,16'bz}),
.i_led16      (i_led16),
.i_led17      (i_led17));
```

Figura 20: Cambio 3 en el archivo `rvfpgasim.v`.

Como el proyecto Vivado se ejecutó sin problemas se asume que no hay errores (si no se detendría la elaboración del bitstream) y se concluye exitosamente la adición de las PTCs del LED tricolor 17.

1.2. Máquina de Estados para un Semáforo

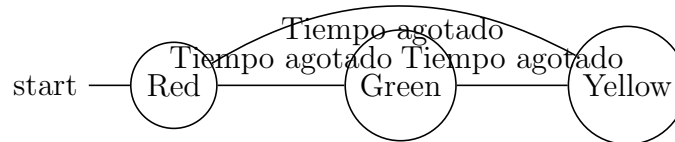
Esta máquina de estados representa el comportamiento de un semáforo de tráfico utilizando tres estados: **Verde (Green)**, **Amarillo (Yellow)** y **Rojo (Red)**.

Cada estado tiene una duración fija y transiciones definidas hacia otros estados.

1.2.1. Estados y Descripciones

- **Green (Verde):** Este estado permite el paso de vehículos. Después de un tiempo predefinido, transiciona al estado **Yellow**.
- **Yellow (Amarillo):** Este estado indica que el semáforo está a punto de cambiar a **Red**. Tiene una duración breve.
- **Red (Rojo):** Este estado detiene el tráfico. Después de un tiempo predefinido, transiciona nuevamente al estado **Green**.

1.2.2. Diagrama de Estados



1.2.3. Código C Pseudodiseñado

El siguiente código muestra cómo se podría implementar esta máquina de estados utilizando un `switch-case` en C:

```

typedef enum {
    STATE_GREEN,
    STATE_YELLOW,
    STATE_RED
} traffic_light_state_t;

traffic_light_state_t currentState = STATE_GREEN;

void UpdateTrafficLight(void) {
    switch (currentState) {
        case STATE_GREEN:
            // Encender LED verde
            if (TimerExpired()) {
                currentState = STATE_YELLOW;
            }
            break;

        case STATE_YELLOW:
            // Encender LED amarillo
            if (TimerExpired()) {
                currentState = STATE_RED;
            }
            break;
    }
}
  
```

```

        case STATE_RED:
            // Encender LED rojo
            if (TimerExpired()) {
                currentState = STATE_GREEN;
            }
            break;
    }
}

```

1.2.4. Explicación del Funcionamiento

1. ****Inicio:**** - El semáforo comienza en el estado **Green**. 2. ****Transición a Yellow:**** - Después de un período predefinido, se detecta que el temporizador ha expirado, y el estado cambia a **Yellow**. 3. ****Transición a Red:**** - El estado **Yellow** dura poco tiempo y luego transiciona a **Red**. 4. ****Ciclo:**** - El estado **Red** espera un período predefinido y luego regresa a **Green**, repitiendo el ciclo.

1.3. Software

Para poder ahora sí llevar a cabo el proyecto en la Nexys, se debe generar un proyecto de Platform IO en VSCode con un programa en C, por lo tanto, se agregan las siguientes imágenes con el código:

1.3.1. Descripción del código

A continuación, se presenta el código con explicaciones detalladas por secciones:

1.3.2. Librerías y definiciones iniciales

Estas líneas incluyen las librerías necesarias para manejar interrupciones y controladores, así como las definiciones de los registros utilizados para displays de 7 segmentos, botones y LEDs tricolores, tanto el 16 como el 17, y un selector de interrupciones.

```

1 // Librerías necesarias para las interrupciones (es normal que no se reconozcan)
2 #include "psp_api.h"
3 #include "bsp_external_interrupts.h"
4 #include "psp_ext_interrupts_ehl.h"
5 #include "bsp_timer.h"
6 #include "bsp_printf.h"
7
8 // Registros para los displays de 7 segmentos
9 #define SegEn_ADDR      0x80001038
10 #define SegDig_ADDR     0x8000103C
11
12 // Registros de los botones y sus interrupciones
13 #define GPIO2_PBs       0x80001800
14 #define GPIO2_INOUT     0x80001808
15 #define RGPIO2_INTE     0x8000180C
16 #define RGPIO2_PTRIG    0x80001810
17 #define RGPIO2_CTRL     0x80001818
18 #define RGPIO2_INTS     0x8000181C

```

Estas definiciones establecen las direcciones de memoria de los periféricos (displays, LEDs, botones) que se utilizarán en el código, el resto se pueden visualizar en el archivo que estaría ubicado en la siguiente ruta: /Proyecto_Semaforo_verilog/RVfpga/Proyecto_Semaforos/src/Proyecto_Semáforos.c.

1.3.3. Enumeración para los estados del semáforo

Se define un `enum` que facilita la gestión de los estados del semáforo:

```

1 // Estados del sem foro, para una ordenada implementaci n de los estados del
2 // sem foro se utiliza un enum
3 typedef enum {
4     STATE_RED,
5     STATE_GREEN,
6     STATE_YELLOW
7 } traffic_light_state_t;
8
9 // Variables globales:
10 traffic_light_state_t currentState = STATE_RED; // Estado inicial
11 int contador7Seg;
12 int i = 0;
```

Esta sección permite que el semáforo funcione mediante estados claramente definidos, mejorando la legibilidad del código.

1.3.4. Función para el manejo del display de 7 segmentos

La siguiente función actualiza el display con números del 0 al 9 y realiza un conteo descendente:

```

1 // Decodificador para los n meros del display de 7 segmentos
2 void SetDisplay(int num) {
3     M_PSP_WRITE_REGISTER_32(SegDig_ADDR, num); // Mostrar el n mero en los displays
4 }
5
6 // Funci n para realizar el desconteo en el display de 7 segmentos
7 void DesconteoDisplay(void) {
8     for (contador7Seg = 9; contador7Seg >= 0; contador7Seg--) {
9         SetDisplay(contador7Seg); // Actualiza el valor del display
10        for (i = 0; i < 10000000; i++); // Delay para hacer visible el conteo
11    }
12 }
```

Esta función realiza una cuenta regresiva en los displays, controlando directamente el hardware.

1.3.5. Manejador de interrupciones

Esta función define qué ocurre cuando se activa la interrupción de un botón, cambiando el estado del semáforo:

```

1 void GPIO_ISR(void) {
2     if(M_PSP_READ_REGISTER_32(RGPIO2_INTS) & 0x1) {
3         // Si se recibe la interrupci n del bot n central:
```

```

4      M_PSP_WRITE_REGISTER_32(HRC_G_17, HRC_VALUE); // Activa Paso del tren.
5      M_PSP_WRITE_REGISTER_32(HRC_R_17, HRC_VALUE_OFF); // Apaga led rojo.
6      (*@dots@*)
7      currentState = STATE_RED; // Reinicia el semforo al estado rojo
8  }
9      M_PSP_WRITE_REGISTER_32(RGPIO2_INTS, 0x0); // Reinicia la interrupci n
10     bspClearExtInterrupt(4); // Limpia el estado pendiente de interrupci n
11 }

```

Esto permite responder a eventos externos, como la presión de un botón.

Antes de acceder a las funciones del estado del semáforo, hay varias funciones que son obligatorias y que no deberían editarse, para así poder realizar las interrupciones, por lo que se agregan pero no son una solución propuesta por los estudiantes, a excepción de la función GPIO_Initialization que es en la que debemos inicializar los registros de las interrupciones de los botones.

```

1 void DefaultInitialization(void)
2 { // Funci n ya suministrada para manejar interrupciones (No editar).
3     u32_t uiSourceId;
4
5     /* Register interrupt vector */
6     pspInterruptsSetVectorTableAddress(&M_PSP_VECT_TABLE);
7
8     /* Set external-interrupts vector-table address in MEIVT CSR */
9     pspExternalInterruptSetVectorTableAddress(G_Ext_Interrupt_Handlers);
10
11     /* Put the Generation-Register in its initial state (no external interrupts are ...
        generated) */
12     bspInitializeGenerationRegister(D_PSP_EXT_INT_ACTIVE_HIGH);
13
14     for (uiSourceId = D_BSP_FIRST_IRQ_NUM; uiSourceId <= D_BSP_LAST_IRQ_NUM; ...
        uiSourceId++)
15     {
16         /* Make sure the external-interrupt triggers are cleared */
17         bspClearExtInterrupt(uiSourceId);
18     }
19
20     /* Set Standard priority order */
21     pspExtInterruptSetPriorityOrder(D_PSP_EXT_INT_STANDARD_PRIORITY);
22
23     /* Set interrupts threshold to minimal (== all interrupts should be served) */
24     pspExtInterruptsSetThreshold(M_PSP_EXT_INT_THRESHOLD_UNMASK_ALL_VALUE);
25
26     /* Set the nesting priority threshold to minimal (== all interrupts should be ...
        served) */
27     pspExtInterruptsSetNestingPriorityThreshold(M_PSP_EXT_INT_THRESHOLD_UNMASK_ALL_VALUE);
28 }
29
30
31 void ExternalIntLine.Initialization(u32_t uiSourceId, u32_t priority, ...
    pspInterruptHandler_t pTestIsr)
32 { // Funci n ya suministrada para manejar interrupciones (No editar).
33     /* Set Gateway Interrupt type (Level) */
34     pspExtInterruptSetType(uiSourceId, D_PSP_EXT_INT_LEVEL_TRIG_TYPE);
35

```

```

36  /* Set gateway Polarity (Active high) */
37  pspExtInterruptSetPolarity(uiSourceId, D_PSP_EXT_INT_ACTIVE_HIGH);
38
39  /* Clear the gateway */
40  pspExtInterruptClearPendingInt(uiSourceId);
41
42  /* Set IRQ4 priority */
43  pspExtInterruptSetPriority(uiSourceId, priority);
44
45  /* Enable IRQ4 interrupts in the PIC */
46  pspExternalInterruptEnableNumber(uiSourceId);
47
48  /* Register ISR */
49  G.Ext_Interrupt_Handlers[uiSourceId] = pTestIsr;
50 }
51
52
53 void GPIO_Initialization(void)
54 { // Aqu se setean los registros de interrupcin de la GPIO2 de los botones.
55   /* Configure GPIO2 interrupts */
56   M.PSP_WRITE_REGISTER_32(RGPIO2_INTE, 0x1F);           /* RGPIO_INTE */
57   M.PSP_WRITE_REGISTER_32(RGPIO2_PTRIG, 0x1F);         /* RGPIO_PTRIG */
58   M.PSP_WRITE_REGISTER_32(RGPIO2_INTS, 0x0);           /* RGPIO_INTS */
59   M.PSP_WRITE_REGISTER_32(RGPIO2_CTRL, 0x1);           /* RGPIO_CTRL */
60 }

```

1.3.6. Estado del semáforo

Las siguientes funciones controlan el comportamiento del semáforo y sus transiciones:

```

1 // Funcion para actualizar los estados del semaforo
2 void TrafficLight(void) {
3     switch (currentState) {
4         case STATE_RED:
5             // Estado rojo: enciende el LED rojo y apaga los dem s
6             M.PSP_WRITE_REGISTER_32(HRC_R_16, HRC_VALUE); // Activa LED rojo
7             M.PSP_WRITE_REGISTER_32(HRC_G_16, HRC_VALUE_OFF); // Apaga LED verde
8             M.PSP_WRITE_REGISTER_32(HRC_B_16, HRC_VALUE_OFF); // Apaga LED azul
9             DesconteoDisplay(); // Llamada al desconteo en cada estado
10            break;
11
12            case STATE_GREEN:
13                // Estado verde: enciende el LED verde y apaga los dem s
14                M.PSP_WRITE_REGISTER_32(HRC_G_16, HRC_VALUE); // Activa LED verde
15                M.PSP_WRITE_REGISTER_32(HRC_R_16, HRC_VALUE_OFF); // Apaga LED rojo
16                M.PSP_WRITE_REGISTER_32(HRC_B_16, HRC_VALUE_OFF); // Apaga LED azul
17                DesconteoDisplay(); // Llamada al desconteo en cada estado
18                break;
19
20            case STATE_YELLOW:
21                // Estado amarillo: enciende el LED amarillo (verde + rojo) y apaga ...
22                // el azul
23                M.PSP_WRITE_REGISTER_32(HRC_R_16, HRC_VALUE); // Activa LED rojo

```

```

23         M_PSP_WRITE_REGISTER_32(HRC_G_16, HRC_VALUE); // Activa LED verde
24         M_PSP_WRITE_REGISTER_32(HRC_B_16, HRC_VALUE_OFF); // Apaga LED azul
25         DesconteoDisplay(); // Llamada al desconteo en cada estado
26         break;
27     }
28 }
29
30 // Funci n que actualiza el estado del sem foro
31 void UpdateState(void) {
32     switch (currentState) {
33         case STATE_RED:
34             // Transici n del estado rojo al verde
35             currentState = STATE_GREEN;
36             break;
37
38         case STATE_GREEN:
39             // Transici n del estado verde al amarillo
40             currentState = STATE_YELLOW;
41             break;
42
43         case STATE_YELLOW:
44             // Transici n del estado amarillo al rojo
45             currentState = STATE_RED;
46             break;
47     }
48 }

```

1.3.7. Funci3n principal

Por 3ltimo, el `main` inicializa las interrupciones, configura los perif3ricos y entra en un bucle infinito para gestionar el sem3foro:

```

1 int main(void)
2 {
3     // Bloque necesario de llamado de funciones para activar interrupci n.
4     /* INITIALIZE THE INTERRUPT SYSTEM */
5     DefaultInitialization(); /* Default initialization */
6     pspExtInterruptsSetThreshold(5); /* Set interrupts threshold ...
   to 5 */
7
8     /* INITIALIZE INTERRUPT LINE IRQ4 */
9     ExternalIntLine.Initialization(4, 6, GPIO_ISR); /* Initialize line IRQ4 ...
   with a priority of 6. Set GPIO_ISR as the Interrupt Service Routine */
10    M_PSP_WRITE_REGISTER_32(Select_INT, 0x3); /* Connect the GPIO ...
   interrupt to the IRQ4 interrupt line */
11
12    /* INITIALIZE THE PERIPHERALS */
13    GPIO.Initialization(); /* Initialize the GPIO */
14    M_PSP_WRITE_REGISTER_32(SegEn_ADDR, 0x0); /* Initialize the 7-Seg ...
   Displays */
15
16    /* ENABLE INTERRUPTS */

```

```

17  pspInterruptsEnable(); /* Enable all interrupts in ...
    mstatus CSR */
18  M_PSP_SET_CSR(D_PSP_MIE_NUM, D_PSP_MIE_MEIE_MASK); /* Enable external ...
    interrupts in mie CSR */
19  // Fin del bloque de llamado de funciones para activar interrupci n.
20
21  // Se habilita el controlador de las PTCs de los LEDs Tricolor:
22  M_PSP_WRITE_REGISTER_32(CTRL_R_16, CTRL_VALUE);
23  M_PSP_WRITE_REGISTER_32(CTRL_G_16, CTRL_VALUE);
24  M_PSP_WRITE_REGISTER_32(CTRL_B_16, CTRL_VALUE);
25  M_PSP_WRITE_REGISTER_32(CTRL_R_17, CTRL_VALUE);
26  M_PSP_WRITE_REGISTER_32(CTRL_G_17, CTRL_VALUE);
27  // Se setean los registros LRC de todos los LEDS Tricolor:
28  M_PSP_WRITE_REGISTER_32(LRC_R_16, LRC_VALUE);
29  M_PSP_WRITE_REGISTER_32(LRC_G_16, LRC_VALUE);
30  M_PSP_WRITE_REGISTER_32(LRC_B_16, LRC_VALUE);
31  M_PSP_WRITE_REGISTER_32(LRC_R_17, LRC_VALUE);
32  M_PSP_WRITE_REGISTER_32(LRC_G_17, LRC_VALUE);
33  // Estado inicial de los LEDs Tricolor:
34  M_PSP_WRITE_REGISTER_32(HRC_R_16, HRC_VALUE_OFF);
35  M_PSP_WRITE_REGISTER_32(HRC_G_16, HRC_VALUE_OFF);
36  M_PSP_WRITE_REGISTER_32(HRC_B_16, HRC_VALUE_OFF);
37  M_PSP_WRITE_REGISTER_32(HRC_R_17, HRC_VALUE);
38  M_PSP_WRITE_REGISTER_32(HRC_G_17, HRC_VALUE_OFF);
39
40  while (1) { // Programa que se ejecuta una y otra vez hasta recibir interrupci n.
41      // Ejecuci n del estado
42      TrafficLight();
43      // Actualiza el semaforo con el estado actual
44      UpdateState();
45  }
46  }

```

2. Conclusiones y Recomendaciones

Se concluye que sí es posible llevar a cabo y ejecutar programas de este tipo, sí poseen una dificultad alta sobretodo en la parte del entendimiento de la arquitectura, hardware y software que se debe implementar en la Nexys, es de suma importancia intentar trazar una línea de trabajo antes de comenzar a desarrollar el proyecto porque puede generar mucha confusión y puntos de retorno complicado si no se comprende bien la arquitectura y hardware de la Nexys. Se recomienda tratar de entender el enlace y sentido que adquieren todos los archivos al explicar su unión y funcionamiento dependiente, y se recomienda además, el dedicarse a estudiar lo suficiente para manejar bien diferentes lenguajes de programación y así explotar todo el potencial de uso de la Nexys y comprender mejor su funcionamiento y composición, como se mencionó anteriormente, es necesario poder manejar bien C/C++ y lenguajes de descripción de hardware, como lo es Verilog, e incluso no solo entender bien la sintaxis e implementación de cada lenguaje, sino estudiar bastante acerca de las estructuras arquitectónicas que utilizan cada uno y como se pueden aplicar. Se concluye además de que la Nexys es complicada de comprender pero al saberla utilizar puede ser muy útil en muchísimas aplicaciones. Exitosamente se logró un programa sólido del funcionamiento de un semáforo con el extra del semáforo del tren.